RABBIT HOLES

Accelerate · Manipal University Jaipur

9

PHASE

Plumbing

What is actually inside .git, and why a commit is not a diff.

Part of **git started**, Accelerate's third workshop.
github.com/accelerate-muj/git-started · MIT licensed.

Contents

1 Porcelain and plumbing

Git's own documentation splits its commands into two groups. **Porcelain** is the friendly layer you use every day: `add`, `commit`, `push`, `log`. **Plumbing** is the low-level machinery underneath: `hash-object`, `cat-file`, `write-tree`, `update-ref`.

You will not use plumbing in daily work. It is worth an hour anyway, because after this phase most of git's behaviour stops being a set of rules to memorise and starts being obvious.

> **WHY THIS EXISTS**
>
> Specific things that stop being confusing once you have read this phase: why branches are cheap, why you cannot lose a commit that was made, why rebase changes hashes, why git is fast at things that ought to be slow, why moving a file is not tracked as a move, and why a merge conflict is genuinely the only situation where git cannot decide for itself.

2 Inside .git

```
cd my-sonnet
ls -a .git
```

```
HEAD          config       description    hooks/
index         info/        logs/          objects/
refs/
```

Item	Contains
`objects/`	Every version of everything, ever. The database.
`refs/`	Branches and tags. Small files holding one hash each.
`HEAD`	Which branch you are on.
`index`	The staging area, as a binary file.
`logs/`	The reflog.
`config`	This repository's settings, including remotes.
`hooks/`	Scripts git runs at certain moments. Phase 11.

That is the whole repository. Copy this directory and you have copied the project including its entire history. Delete it and you have an ordinary folder.

3 Four kinds of object

The object database stores exactly four things.

blob The contents of a file. Just bytes. No name, no path, no permissions.

tree A directory listing. Names, modes, and the hash of the blob or tree each name points to.

commit A pointer to one tree, plus parents, author, committer, timestamps, and a message.

tag An annotated tag: a pointer to an object, plus a name, tagger and message.

Every object is stored under the SHA of its own contents. This is the central idea, and everything else follows from it.

3.1 Watch it happen

TRY IT

```
mkdir plumbing-demo && cd plumbing-demo && git init
find .git/objects -type f
```

Nothing. Now store some content directly, bypassing `git add` entirely:

```
echo "Shall I compare thee" | git hash-object -w --stdin
```

Git prints a hash and writes one object. Look:

```
find .git/objects -type f
```

```
.git/objects/8b/13727ee5c4b4b1e0e13a34e30f0b4d0b34e5a2
```

The first two characters of the hash are the directory, the rest is the filename. That is only to avoid putting a million files in one directory.
Read it back:

```
git cat-file -p 8b13727
git cat-file -t 8b13727
git cat-file -s 8b13727
```

`-p` prints the contents, `-t` the type, `-s` the size.

3.2 The hash is the content

Store the same text twice and you get the same hash, and the second write does nothing because the object already exists.

This is what **content addressed** means, and it has consequences that show up everywhere:

- **Deduplication is automatic.** A file unchanged across a thousand commits is stored once. Copy a file and both paths point at the same blob.
- **Integrity is automatic.** If a byte in `.git/objects` is corrupted, its contents no longer hash to its filename. `git fsck` finds it.
- **History cannot be quietly edited.** Change any commit and its hash changes, so every descendant's hash changes. That is not a security feature bolted on, it is a side effect of the storage design.
- **Rebasing must produce new hashes.** A rebased commit has a different parent, so different content, so a different hash. There is no way around it.

3.3 A commit is a snapshot, not a diff

This is the most common misconception about git, including among people who use it daily.

A commit does not store what changed. It stores a pointer to a tree, and the tree describes the *entire* project at that moment.

```
git add . && git commit -m "First"
git cat-file -p HEAD
```

```
tree 92b8a1c4f7e3d2b5a8c1f4e7d0b3a6c9f2e5d8b1
author Vidhu <physicsexplore2023@gmail.com> 1756054800 +0530
committer Vidhu <physicsexplore2023@gmail.com> 1756054800 +0530

First
```

There is no diff anywhere in there. Follow the tree:

```
git cat-file -p 92b8a1c
```

```
100644 blob 8b13727ee5c4b4b1e0e13a34e30f0b4d0b34e5a2    sonnet.md
040000 tree 3f2a1b9c8d7e6f5a4b3c2d1e0f9a8b7c6d5e4f3a    poems
```

Names and permissions live in the tree, not the blob. A blob is anonymous content.

> **DEEPER**
>
> Two things follow from this.
> **Git does not track renames.** It cannot, because there is nothing to track. When you move a file, the blob is unchanged and the tree lists it under a new name. When `git log --follow` or `git diff -M` reports a rename, it is *detecting* one after the fact by noticing that a blob disappeared from one path and an identical or similar one appeared at another. This is why renames are sometimes detected and sometimes not, and why the similarity threshold is configurable.
> **Every diff you have ever seen was computed on demand.** `git show`, `git log -p`

> and every diff on GitHub are generated at the moment you ask, by comparing two trees. Git stores snapshots and calculates differences. Most other version control systems did the opposite.

4 Refs, and what a branch really is

```
cat .git/refs/heads/main
```

```
a1b2c3d4e5f67890abcdef1234567890abcdef12
```

That is the entire branch. Forty-one bytes, a hash and a newline.

> **WHY THIS EXISTS**
>
> This is why branching in git is instant and free, and why it was a revelation in 2005. In older systems a branch meant copying the whole tree on the server, so branches were heavyweight things you asked permission for and created rarely.
> In git, creating a branch writes forty-one bytes. Deleting one removes forty-one bytes, which is also why deleting a branch never deletes any commits: you removed a label, not the thing it pointed at. Phase 4's branch recovery works because of this sentence.

```
cat .git/HEAD
```

```
ref: refs/heads/main
```

`HEAD` normally points at a branch, which points at a commit. When it contains a raw hash instead, you are in **detached HEAD** state, which is all that condition is: `HEAD` pointing directly at a commit with no branch in between. Commits you make there are real, but nothing references them, so they are only findable through the reflog.

```
ls .git/refs/heads .git/refs/tags .git/refs/remotes
git show-ref
git update-ref refs/heads/experiment a1b2c3d
git symbolic-ref HEAD
```

> **NOTE**
>
> Once you have many refs, git packs them into `.git/packed-refs` and the individual files disappear. `git show-ref` reads both, so use it rather than `ls`.

5 Building a commit by hand

Worth doing once. It removes the last of the mystery.

TRY IT

```
mkdir by-hand && cd by-hand && git init
```

Make a blob:

```
echo "line one" | git hash-object -w --stdin
```

Put it in the index under a name:

```
git update-index --add --cacheinfo 100644,<blob-sha>,poem.txt
```

Turn the index into a tree:

```
git write-tree
```

Make a commit pointing at that tree:

```
echo "Made by hand" | git commit-tree <tree-sha>
```

Point a branch at it:

```
git update-ref refs/heads/main <commit-sha>
```

Now:

```
git log
git status
```

An ordinary repository with an ordinary commit, and `git commit` was never run. That command is a convenience wrapper around exactly these five steps.

6 The index

`.git/index` is a binary file listing every tracked path with its blob hash, permissions, and cached filesystem metadata: size, modification time, inode.

That cache is why `git status` is fast. To decide whether a file changed, git compares stat information rather than reading and hashing every file in the project. Only when the metadata differs does it do real work.

```
git ls-files -s
git ls-files -s --debug
```

It also explains a class of confusing behaviour: if you restore a file from a backup and the timestamp changes, git may consider it modified even when the bytes are identical. And `git status` being slow on a huge repository is usually this cache being defeated, which is what `core.fsmonitor` exists to fix.

7 Packfiles

Storing every version as a separate compressed file would waste space on a project with long history. Periodically git repacks loose objects into a **packfile**, where similar objects are stored as deltas against each other.

```
git count-objects -vH
git gc
git verify-pack -v .git/objects/pack/pack-*.idx | tail -5
```

> **DEEPER**
>
> The delta compression here is not the same as storing diffs. Objects are still content addressed snapshots, and any object can be reconstructed independently. The deltas are purely a storage optimisation inside the pack, chosen by heuristics about which objects resemble each other, and they can even be between unrelated files.
> This is how a repository with a decade of history and hundreds of thousands of commits can clone in a few hundred megabytes. It is also why `git gc` sometimes shrinks a repository dramatically: loose objects that were stored individually get packed and deltified together.

`git gc` runs automatically when loose objects accumulate. It also prunes unreachable objects older than the grace period, which is the ninety day figure from Phase 4.

8 The commit graph

Every commit records its parents. Follow those pointers and you get a directed acyclic graph.

- **Directed** because each commit points backwards to its parents, never forwards. A commit has no idea what came after it.
- **Acyclic** because a commit's hash includes its parents' hashes, so a cycle would require a hash to contain itself.

Ordinary commits have one parent. Merge commits have two or more. The very first commit has none.

```
git rev-list --all --count
git rev-parse HEAD
git rev-parse HEAD~2
git rev-parse main@{yesterday}
git cat-file --batch-all-objects --batch-check
```

`git rev-parse` is the general tool for turning any expression into a hash. Every place git accepts a commit, it accepts these forms:

Expression	Means
`HEAD~3`	Three first-parent steps back
`HEAD^2`	The second parent, on a merge commit
`main@{2.days.ago}`	Where `main` pointed then, via the reflog
`HEAD@{5}`	Five reflog entries ago
`main..feature`	Reachable from feature, not from main
`main...feature`	Reachable from either, not from both
`:/fix the parser`	The newest commit whose message matches

> **TRAP**
>
> `~` and `^` are different and the difference only shows up at merge commits. `~n` walks *n* steps back always following the first parent. `^n` selects *which* parent, without moving further. So `HEAD~2` is two commits back, and `HEAD^2` is the second parent of the current commit. On a linear history they are easy to confuse because `HEAD~1` and `HEAD^1` mean the same thing.

9 SHA-1 and what replaces it

Git has used SHA-1 since 2005, and SHA-1 has been considered cryptographically broken since the SHAttered collision in 2017.

This matters less than it sounds, for two reasons. Git uses hashing mainly for integrity and addressing rather than as a security boundary, and since 2018 it ships with a hardened SHA-1 that detects the known collision attack and refuses.

The real fix is SHA-256 support, which exists:

```
git init --object-format=sha256
```

> **NOTE**
>
> Do not use it yet for anything shared. SHA-256 repositories cannot interoperate with SHA-1 ones, and GitHub does not support them. The migration path is designed and partially built, and the ecosystem has not moved. Know that it exists and that the forty

> character hashes you see today are not permanent.

10 Reference

Command	Does
`git cat-file -p <sha>`	Print any object
`git cat-file -t <sha>`	Its type
`git hash-object -w <file>`	Store a file as a blob
`git ls-tree HEAD`	List a tree
`git ls-files -s`	Show the index
`git rev-parse <expr>`	Resolve anything to a hash
`git show-ref`	Every ref, packed or loose
`git update-ref <ref> <sha>`	Move a ref by hand
`git write-tree`	Index into a tree object
`git commit-tree <tree>`	Tree into a commit object
`git count-objects -vH`	Repository size
`git fsck`	Check integrity, find orphans
`git gc`	Repack and prune

> **CHECK YOURSELF**
>
> 1. Name the four object types and what each stores.
> 2. A commit stores a tree, not a diff. Give two consequences.
> 3. How many bytes is a branch, and what does that imply about deleting one?

4. Why must rebasing change commit hashes?
5. What is the difference between `HEAD~2` and `HEAD^2`?
6. Where does a file's *name* live: blob, tree, or commit?

Next: Phase 10, *Trust*, on signing commits so people can prove you wrote them, tagging releases, and what to do the day someone commits a password.